```
1RXY  direct
3RXY  indirect
&a     address
*p      value
```



159.102 · COMPUTER SCIENCE

— WEEK 6 —

Pointers & Call by Reference

Exam revision notes — everything from Week 6 in one quick peek. Simple English, simple pictures, zero fluff.

```
int *ptr = &a;   // the whole week in one line
```

01 · FOUNDATION

Memory Addresses — the Big Picture

Every variable you declare is a **box in memory**, and every box has an **address** — like houses on a street. The name (`a` , `b`) is for humans; the address is what the computer actually uses. When you write `int a, b;` the system sets aside two boxes and remembers where each one lives.



You already met this idea in Week 5 machine code — the MASSEY machine had two ways to load a register, and Week 6 is the same idea in C++:

Instruction	What it does	Name
`1RXY`	Load register R with the **actual number** XY	Direct load — "here is the pizza 🍕"
`3RXY`	Load register R with the number **found at address** XY	Indirect load — "here is the address 📍"

```
// seeing values vs addressesint a, b;
a = 3;  b = 5;
cout << a;    // 3        → the VALUE
cout << &a;   // 0x7db6… → the ADDRESS (hexadecimal)
```

💡 **ONE-LINER**

Every variable is a box; every box has an address; `&` asks "where do you live?"

02 · THE TWO OPERATORS

& and * — the Only Two Symbols That Matter

Week 6 really comes down to two operators used in opposite directions. If you have a **variable** and want its address, use `&` . If you have an **address** and want the value stored there, use `*` .

📍 **& — "THE ADDRESS OF"**

`&x` = the address of variable x. `&a` gives something like `0x7db6` . Read it out loud: **"address of a"**.

🚶 *** — "THE VALUE AT"**

`*ptr` = the value stored at that address — **"walk to the house and look inside"**. Also called the **indirection** or **dereference** operator.

The star has two jobs — don't mix them up

Where you use *	Meaning	Example
When **declaring**	"make a pointer"	`int *ptr;`
When **using** it	"the value at this address"	`*ptr` → gives 17

```
// both operators togetherint a = 21;
cout << &a;       // the address of a
cout << *(&a);   // the value at (the address of a) → 21
```

03 · CORE CONCEPT

Pointers & the NULL Pointer

A **pointer is a variable that stores a memory address as its value.** A normal variable holds a number like 17; a pointer holds an address like 32186. You can declare a pointer to any data type — `char *p;` or `float *r;` — and you must load it with an **address**, never an ordinary number.

```
// a pointer in actionint k = 17;
int *ptr;         // pointer to an int (can be empty for now)
ptr = &k;           // ptr now holds k's address

cout << k;       // 17         → the value
cout << ptr;      // 0x…        → the address of k
cout << *ptr;     // 17         → walk there, read inside

ptr = 3;           // ❌ ERROR — 3 is a number, not an address
```



NULL — pointing at "nothing"

Sometimes a pointer has no address yet, so we set it to `NULL` to officially declare "this pointer is empty". The golden rule: **fine to store, never look inside**. Using `*` on a NULL pointer tells the computer "walk to address… which one?" — and the program crashes. The notes call this "a BAD program" for a reason.

```
int *ptr;
ptr = NULL;        // OK — pointer exists, points to nothing
cout << *ptr;     // ❌ CRASH — walk to which house? 💥
```

> 💡 **ANALOGY**
>
> A pointer is a **sticky note 📇 with an address written on it**. NULL = a sticky note that says "no address written yet". Never try to visit the house on a blank note.

04 · THE MAGIC TRICK

Indirection & Declaration Traps

Indirection means you can change a variable without ever using its name — exactly like the machine code `3RXY` load. Point the pointer at a box, then use `*ptr = value;` to write straight into that box. The cout below looks totally normal — the surprise is that `x = 12;` was never written anywhere.

```
// the trick: x becomes 12 without being namedint x;
int *ptr;
ptr = &x;
*ptr = 12;          // find the box whose address is in ptr → put 12 in it
cout << x;         // 12  😲 — we loaded x INDIRECTLY
```

Definition (memorise)	Instruction
Load a value directly	`x = 12;`
Load a value indirectly	`*ptr = 12;` — after setting `ptr = &x;`

The two directions of `*ptr`: on the **left** of `=` it **WRITES** into the box at that address (`*ptr = 12;`); inside a cout it **READS** what is there (`cout << *ptr;`).

⚠️ The two-pointer trap

The star belongs to the **name, not the type**. Every pointer needs its own star, so be careful when declaring two at once — lazy star placement silently changes what the second variable is:

```
int *ptr1, *ptr2;   // ✅ two pointers — each has its own starint* ptr1, ptr2;
// ❌ TRAP — ptr1 is a pointer, ptr2 is a plain int!
```

> 💡 **ANALOGY**
>
> `x = 12;` = call the family by name: “Mr. x, here’s your pizza 🍕”. `*ptr = 12;` = give the delivery guy an address: “drop it at house #32186”. Same pizza, same house — **no name needed, only the address**.

05 · THE FAMOUS FAILURE

Call by Value — the Swop That Refuses to Work

`swop` is not built-in — it is a function **we write ourselves** (the name just means “swap”). The version below looks textbook-perfect, the logic runs fine… and yet **nothing changes**. This is the most important “why” question of the week.

```
void swop(int x, int y) {
  int temp;
  temp = x;  x = y;  y = temp;   // swaps the copies perfectly ✅
}

int main() {
  int a = 3, b = 5;
  swop(a, b);
  cout << a << b;      // STILL 3 and 5 🤯 — nothing changed
}
```

The reason: C++ uses **call by value** — the function does **not** receive your real variables. It receives **photocopies of the values**. The parameters (`x` , `y`) are temporary local boxes: copies are made at call time, the function happily swaps the copies, and when it ends the copies are destroyed. Your real boxes in main never even noticed.

MAIN (real)	—— copy values ——▶️	FUNCTION (copies)
a=3 b=5	only numbers cross over	**x=3 y=5**
untouched originals		swapped, then destroyed 🗑️

📄 **WHY C++ DOES THIS**

Call by value lets you send a **calculation** — there is no “real variable” to send: `swop(a + 14 % 2, b - 3);` works fine.

↩️ **WHY IT’S NOT ENOUGH**

`return` can bring back only **ONE** result. Swop needs to change **TWO** variables — one return can’t do it.

💡 **ANALOGY**

It’s like sending a **photo on WhatsApp** 📱 — your friend can draw all over their copy, your original notebook stays untouched. The function never had your notebook, only a picture of it.

06 · THE TWO FIXES

Make the Function Reach the Real Boxes

The problem was never the swap logic — it was that the function only ever saw **copies**. C++ will never let you send the real boxes, but you **can send their addresses**, and with an address the function can reach the real boxes from far away. There are two ways to do it.

Fix 1 — pass addresses (call by value + pointers)

```
void swop(int *ptr1, int *ptr2) {   // parameters are POINTERSint temp;
  temp  = *ptr1;      // walk to a's real box, read the 3
  *ptr1 = *ptr2;      // walk to a's real box, write 5 into it
  *ptr2 = temp;       // walk to b's real box, write 3 into it
}

// in main:
swop(&a, &b);      // ✅ send the ADDRESSES — now prints 5 3
```

> 💡 **WHY COPYING DOESN'T RUIN IT**
>
> It is still call by value — C++ still copies! But copying a **value** is useless, while copying an **address** still points at the real box. Like a **Google Maps pin** 📍 — copy the pin 100 times, every copy leads to the same real house. Notice: the function changes `a` and `b` **without even knowing their names** — pure indirection.

Fix 2 — call by reference (the clean way)

```
void swop(int &x, int &y) {    // ← the ONLY change: & in the parametersint temp;
  temp = x;  x = y;  y = temp;    // plain code — no stars anywhere
}

// in main:
swop(a, b);          // ✅ normal call — prints 5 3. One character fixed it!
```

`int &x` means **x is not a copy — it is a nickname for the real variable a**. Your name is Alexander, your friend calls you Alex — two names, one person. If Alex gets a haircut, Alexander got a haircut. So inside the function, `x = y;` changes **a itself**, because x IS a.

> 🤫 **THE SECRET (EXAM FAVOURITE)**
>
> Call by reference **is actually using pointers — C++ just hides them from us**. Behind the scenes it passes the address and does all the star work automatically. The notes' advice: if you want a function that changes values back in main, **use call by reference**.

07 · THE SCOREBOARD

Three Ways to Pass Parameters — Side by Side

	Call by value	Value + pointers	Call by reference
Header	`swop(int x, int y)`	`swop(int *p1, int *p2)`	`swop(int &x, int &y)`
Call it with	`swop(a, b)`	`swop(&a, &b)`	`swop(a, b)`
Function receives	copies 📄	addresses 📍	nicknames 🧑
Changes main's vars?	❌ never	✅ yes	✅ yes
Style	simple but weak	works, but messy 😖	clean & easy 😍

Pointer vs Reference — the full comparison

	Pointer 📍	Reference 🧑
Declare	`int *p;` — may stay empty for now	`int &r = a;` — **must** attach immediately
Read / write value	`*p` — star needed (walk there)	`r` — just the name (you are already "there")
Point elsewhere later?	✅ `p = &b;` re-points freely	❌ never — `r = b;` copies b's **value** into a (sneaky!)
Can be NULL?	✅ `p = NULL;`	❌ never
Own address?	✅ `&p` = the pointer's own box	❌ `&r` = just a's address
What it really is	a real box that stores an address	an alias — a second name, no box of its own

> 💡 **ONE-LINER**
>
> Pointer = **"I know where they live"** (needs `*` to visit). Reference = **"I AM them"** (use it directly, like a normal variable).

08 · EXAM ARMOUR

Invalid Lines, Memory Hooks & Self-Test

These are the lines examiners love, because each one looks almost right. For every line, be able to say the **reason** out loud — that is exactly how short-answer questions are marked.

Line	Why it fails
`int &c = 4;` ❌	A nickname must be a nickname **for a real variable** — 4 has no house. Use `int &c = a;` ✅
`&c = 3;` ❌	"Address of c" is **read-only** — you cannot rewrite where a variable lives. Only `c = 3;` or `int *p = &c;` ✅
`*c = 4;` (c is a reference) ❌	`*` only works on **pointers** — a reference holds no address, nothing to walk through. Just write `c = 4;` ✅
`ptr = 3;` ❌	A pointer needs an **address**, not an ordinary number. Use `ptr = &k;` ✅
`int* p1, p2;` ❌	p2 is a **plain int** — every pointer needs its own star. Use `int *p1, *p2;` ✅
`int &r2;` ❌	A reference **cannot be born empty** — attach it to a variable on the same line.

Memory hooks 🧠

🏠 **Variable** = a house; its **address** = the house number. `&` asks where it lives.

📇 **Pointer** = sticky note with an address. `*` = walk there and look inside.

🧑 **Reference** = nickname. Alex IS Alexander — one person, two names, used without stars.

📄 **Call by value** = photocopy. Doodle on the copy all you like — the original stays clean.

📍 Copied **address** = Google Maps pin. Every copy of the pin leads to the same real house.

💥 `*` on **NULL** = walking to "nowhere" — the program crashes.

Self-test — predict the output 🎯

`Q1` `int a = 3; int *p = &a; *p = 7;` — what is `a` now?

`Q2` A call-by-value `swop(a, b)` does `x = 100;` inside. After it returns, what is `a`?

`Q3` In `int *p1, p2;` — what exactly is `p2`?

`Q4` `ptr = NULL; cout << *ptr;` — what happens?

`Q5` `void f(int &n){ n = 9; }` called as `f(a);` — what happens to `a`?

`Q6` Name the **only** difference between the broken swop and the working one.

ANSWER KEY

A1 — 7. `*p = 7;` writes through the address into a's box. **A2** — still 3. x was a photocopy; it died when the function ended. **A3** — an ordinary int; the star before p1 doesn't cover p2. **A4** — crash 💥; never dereference NULL. **A5** — a becomes 9; n is a nickname for a, so changing n changes the real box. **A6** — the `&` in the parameters: `int &x, int &y`.